

Transformações no API Gateway

Request/Response Transformations e Mapping Templates

AWS Certified Developer Associate

Instrutor: Marlon Anesi

O que são Transformações?

Definição

Processo de modificar requisições (request) antes de enviar ao backend e respostas (response) antes de retornar ao cliente.

Por que usar?

- Adaptar formato entre cliente e backend
- Adicionar/remover/modificar headers
- Transformar JSON para XML ou vice-versa
- Extrair parâmetros de query string ou path
- Mapear códigos de status HTTP

Onde aplicar?

- Method Request: validação antes de integração
- Integration Request: transformar para backend
- Integration Response: transformar resposta backend
- Method Response: formatar resposta final

Mapping Templates com Velocity

O que são Mapping Templates?

Scripts escritos em Velocity Template Language (VTL) que transformam requisições e respostas.

Contexto Disponível

- `$input`: dados da requisição (body, path, query, headers)
- `$context`: metadados (requestId, stage, identity)
- `$stageVariables`: variáveis do stage
- `$util`: funções utilitárias (parseJson, urlEncode)

Exemplo: Extrair query string

```
#set($name = $input.params('name'))  
{ "userName": "$name" }
```

Exemplo: Transformar response

```
#set($inputRoot = $input.path('$'))  
{ "status": "success", "data": $inputRoot }
```

Content-Type e Passthrough

Content-Type Mapping

Mapping templates são aplicados com base no Content-Type:

- application/json: para requisições JSON
- application/xml: para requisições XML
- text/plain: para texto simples

Passthrough Behavior

Define o que fazer quando não há mapping template para o Content-Type:

- WHEN_NO_MATCH: passa requisição sem transformação
- WHEN_NO_TEMPLATES: passa se não houver templates
- NEVER: rejeita com erro 415 (Unsupported Media Type)

Importante para Exame

- NEVER é mais restritivo e seguro
- WHEN_NO_MATCH é mais flexível

Casos de Uso e Boas Práticas

Casos de Uso Comuns

- Adaptar API antiga para novo formato
- Converter JSON para XML
- Adicionar campos calculados
- Remover campos sensíveis
- Mapear erros para formatos padronizados

Alternativas

Para transformações complexas:

- Use Lambda para lógica complexa
- Velocity é limitado e verboso
- Lambda é mais flexível e testável

Boas Práticas

- Teste mapping templates com dados reais antes de deploy
- Use passthrough NEVER para APIs críticas (validação estrita)
- Documente transformações no código ou README
- Prefira Lambda para lógica complexa



Pontos-chave para Revisão

1. Transformações

- Modificam requisições e respostas
- Aplicadas em 4 pontos: Method Request/Response, Integration Request/Response

2. Mapping Templates (VTL)

- Velocity Template Language
- Variáveis: \$input, \$context, \$stageVariables, \$util

3. Passthrough Behavior

- WHEN_NO_MATCH: passa sem transformar
- NEVER: rejeita com erro 415
- Use NEVER para validação estrita

LEMBRE-SE: Se menciona 'transformar formato' ou 'mapping' > pense Velocity Templates! Se precisa lógica complexa > use Lambda!